浙江大学

本科实验报告

课程名称： 计算机网络基础

实验名称： 基于 Socket 接口实现自定义协议通信

姓　　名： 姜快快

学　　院： 计算机学院

专　　业： 计算机科学与技术

学　　号： 3210104669

指导教师：

2026 年 10 月 12 日

浙江大学实验报告

实验名称： 基于 Socket 接口实现自定义协议通信

实验类型： 编程实验

同组学生：

实验地点： 计算机网络实验室

1 实验目的

掌握 Socket 编程接口编写基本的网络应用软件。

2 实验内容

根据自定义的协议规范，使用 Socket 编程接口编写基本的网络应用软件。

- 掌握 C 语言形式的 Socket 编程接口用法，能够正确发送和接收网络数据包
- 开发一个客户端，实现人机交互界面和与服务器的通信
- 开发一个服务端，实现并发处理多个客户端的请求
- 程序界面不做要求，使用命令行或最简单的窗体即可

2.1 功能要求

2.1.1 运输层协议

运输层协议采用 TCP。

2.1.2 客户端功能

客户端采用交互菜单形式，用户可以选择以下功能：

1. **连接**：请求连接到指定地址和端口的服务端
2. **断开连接**：断开与服务端的连接
3. **获取时间**：请求服务端给出当前时间
4. **获取名字**：请求服务端给出其机器的名称

5. **活动连接列表**：请求服务端给出当前连接的所有客户端信息（编号、IP 地址、端口等）
6. **发消息**：请求服务端把消息转发给对应编号的客户端，该客户端收到后显示在屏幕上
7. **退出**：断开连接并退出客户端程序

2.1.3 服务端功能

服务端接收到客户端请求后，根据客户端传过来的指令完成特定任务：

- 向客户端传送服务端所在机器的当前时间
- 向客户端传送服务端所在机器的名称
- 向客户端传送当前连接的所有客户端信息
- 将某客户端发送过来的内容转发给指定编号的其他客户端

2.1.4 编程要求

- 采用异步多线程编程模式，正确处理多个客户端同时连接、同时发送消息的情况
- 本实验涉及到网络数据包发送部分不能使用任何的 Socket 封装类，只能使用最底层的 C 语言形式的 Socket API
- 服务端监听端口使用学号后四位：**4669**

3 主要仪器设备

- 联网的 PC 机
- Windows 操作系统
- MinGW C++ 编译器（支持 C++17 标准）
- Winsock2 库

4 操作方法与实验步骤

4.1 协议设计

本实验采用自定义应用层协议，报文格式如下：

字段	长度	说明
type	4 字节	消息类型（网络字节序）
request_id	4 字节	请求 ID（网络字节序）
body_length	4 字节	消息体长度（网络字节序）
body	变长	消息体内容（UTF-8 编码）

报头共 12 字节，所有整数采用网络字节序（大端序）。消息类型定义如下：

消息类型	类型值
请求时间	1
请求名字	2
请求客户端列表	3
请求发送消息	4
响应时间	101
响应名字	102
响应客户端列表	103
转发消息	104
消息转发确认	105
问候消息	106
错误响应	199

4.2 客户端编写步骤

客户端采用多线程模式，包括主线程和接收线程。

4.2.1 主线程职责

1. 运行初始化，调用 `WSAStartup()` 和 `socket()`
2. 显示菜单功能，列出 7 个选项
3. 等待用户选择并执行相应操作
4. 处理接收线程放入消息队列的数据
5. 显示响应消息和转发消息

4.2.2 接收线程职责

1. 循环调用 `recv()`，接收服务端数据
2. 拼接并解析完整数据包

3. 将响应消息或指示消息放入线程安全消息队列
4. 收到主线程退出通知后结束

4.3 服务端编写步骤

服务端采用多线程模式，主线程负责接受连接，每个客户端由独立的工作线程处理。

4.3.1 主线程职责

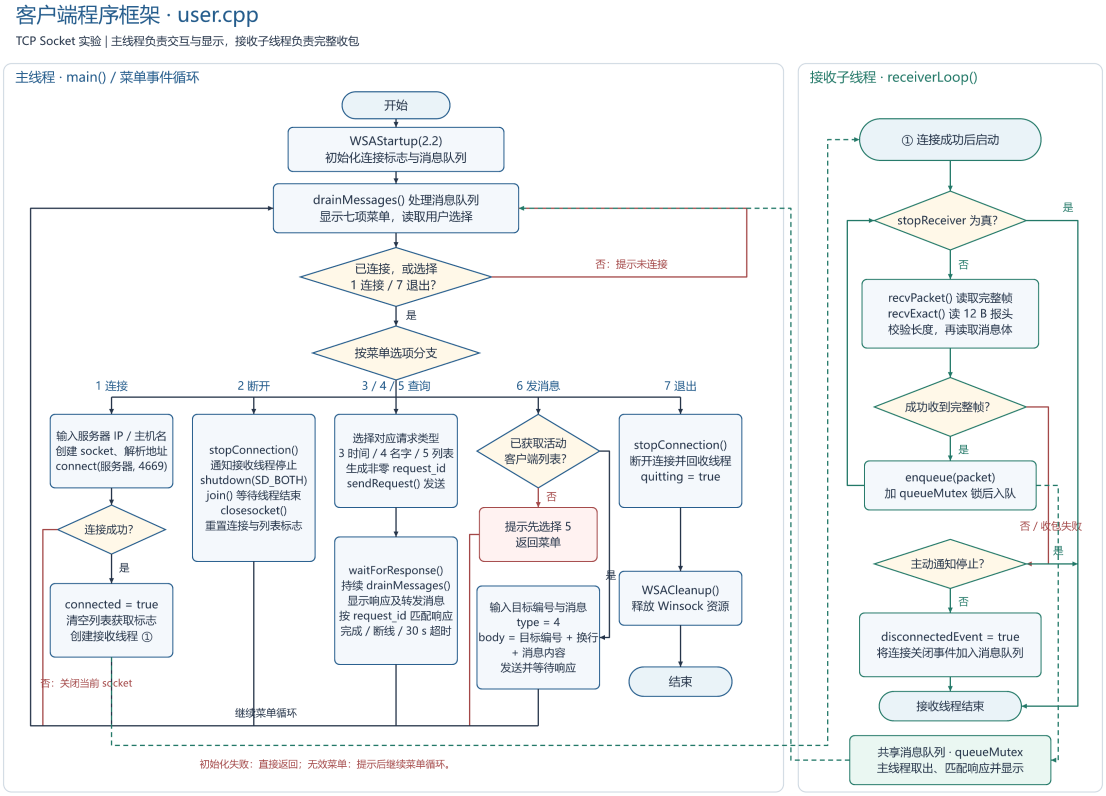
1. 初始化 Winsock，创建监听 Socket
2. 绑定学号后四位端口（4669）
3. 调用 `listen()` 设置连接等待队列
4. 循环调用 `accept()` 接受客户端连接
5. 为每个客户端创建处理线程

4.3.2 工作线程职责

1. 发送问候消息（可选）
2. 循环调用 `recv()`，接收客户端请求
3. 根据请求类型执行相应操作：
    - 获取时间：调用 `time()` 并返回
    - 获取名字：调用 `GetComputerNameW()` 并返回
    - 获取客户端列表：返回所有活动客户端信息
    - 转发消息：将消息转发给目标客户端
4. 连接断开或异常时清理资源

5 实验数据记录和处理

5.1 客户端和服务器端框架图



主线程显示：时间(101)、名字(102)、列表(103)、转发消息(104)、发送确认(105)、欢迎(106)、错误(199)。
断线事件由主线程处理：connected = false，clientListFetched = false；等待输入时也持续处理队列。

共用协议：12 字节网络序报头 [type | request_id | body_length] + UTF-8 消息体；sendAll / recvExact 处理部分收发。

实线：控制流程 绿色虚线：线程启动 / 消息入队 依据：repo/Lab2/user.cpp；lab2/report2-socket.docx

图 1: 客户端程序框架图

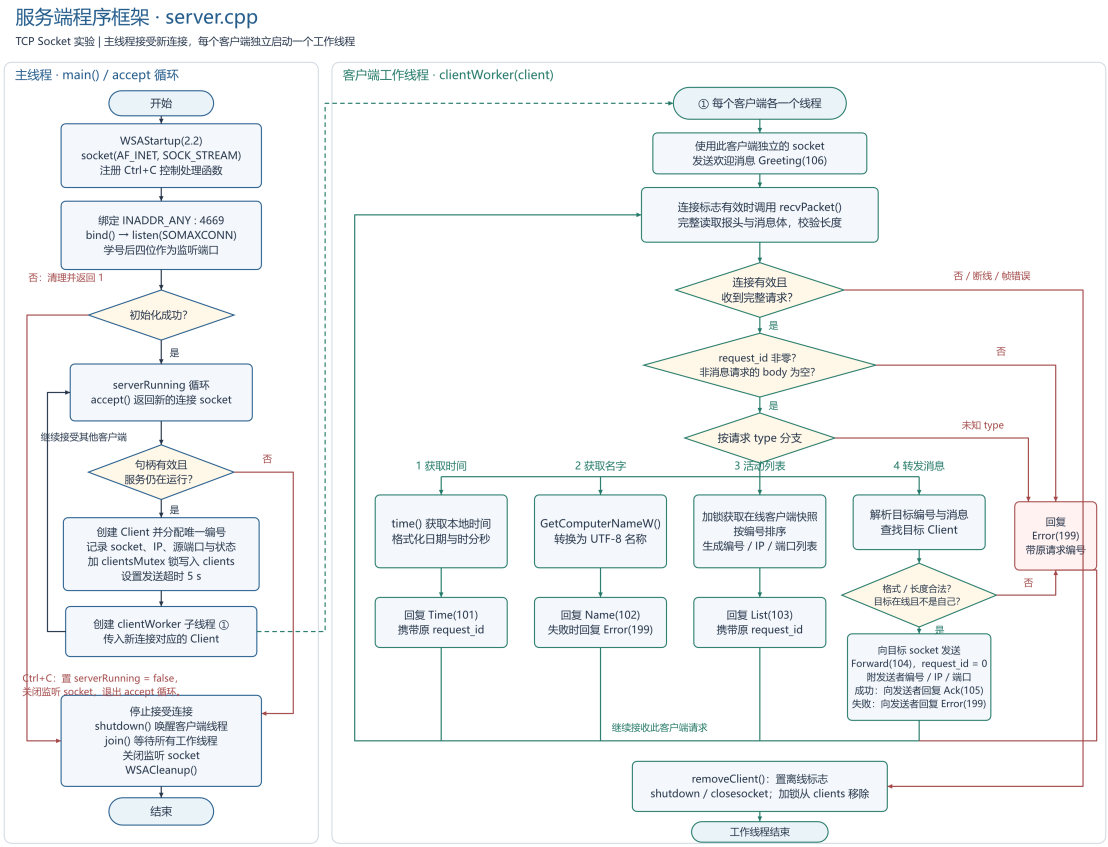

服务端程序框架 · server.cpp
TCP Socket 实验 | 主线程接受新连接，每个客户端独立启动一个工作线程
主线程 · main() / accept 循环
开始
WSAStartup(2.2)
socket(AF_INET, SOCK_STREAM)
注册 Ctrl+C 控制处理函数
绑定 INADDR_ANY : 4669
bind() → listen(SOMAXCONN)
学号后四位作为监听端口
否：清理并返回 1
初始化成功?
是
serverRunning 循环
accept() 返回新的连接 socket
继续接受其他客户端
句柄有效且
服务仍在运行?
否
是
创建 Client 并分配唯一编号
记录 socket、IP、源端口与状态
加 clientsMutex 锁写入 clients
设置发送超时 5 s
创建 clientWorker 子线程 ①
传入新连接对应的 Client
Ctrl+C：置 serverRunning = false，
关闭监听 socket，退出 accept 循环
停止接受连接
shutdown() 唤醒客户端线程
join() 等待所有工作线程
关闭监听 socket
WSACleanup()
结束
客户端工作线程 · clientWorker(client)
① 每个客户端各一个线程
使用此客户端独立的 socket
发送欢迎消息 Greeting(106)
连接标志有效时调用 recvPacket()
完整读取报头与消息体，校验长度
连接有效且
收到完整请求?
否 / 断线 / 帧错误
是
request_id 非零?
非消息请求的 body 为空?
否
是
按请求 type 分支
未知 type
1 获取时间
2 获取名字
3 活动列表
4 转发消息
time() 获取本地时间
格式化日期与时分秒
GetComputerNameW()
转换为 UTF-8 名称
加锁获取在线客户端快照
按编号排序
生成编号 / IP / 端口列表
解析目标编号与消息
查找目标 Client
回复
Error(199)
带原请求编号
格式 / 长度合法?
目标在线且不是自己?
否
是
回复 Time(101)
携带原 request_id
回复 Name(102)
失败时回复 Error(199)
回复 List(103)
携带原 request_id
向目标 socket 发送
Forward(104)，request_id = 0
附发送者编号 / IP / 端口
成功：向发送者回复 Ack(105)
失败：向发送者回复 Error(199)
继续接收此客户端请求
removeClient()：置离线标志
shutdown / closesocket；加锁从 clients 移除
工作线程结束
并发保护：clientsMutex 保护客户端注册表；每个 Client 的 sendMutex 保证响应与转发帧不会交错。
共用协议：12 B 网络序报头 + UTF-8 消息体（≤ 64 KiB）；sendAll / recvExact 循环处理部分收发。
依据：repo/Lab2/server.cpp；lab2/report2-socket.docx | 工作线程只使用 accept 返回的客户端 socket。

图 2: 服务端程序框架图

5.2 客户端初始运行菜单

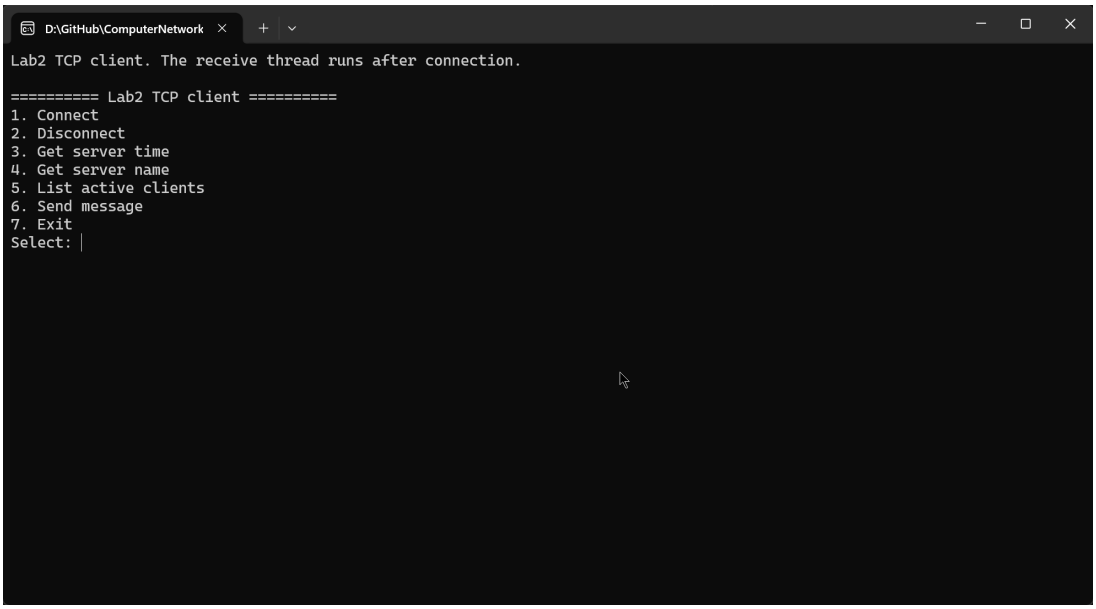


图 3: 客户端初始运行菜单

客户端启动后显示 7 个菜单选项，包括连接、断开连接、获取时间、获取名字、列出活动客户端、发送消息和退出。未连接时只能选择连接和退出功能。

5.3 客户端接收数据子线程关键代码

客户端接收线程的核心代码如下（详见附录 A）：

```
void receiverLoop() {
    Packet packet;
    while (!stopReceiver.load() && recvPacket(clientSocket,
        packet)) {
        enqueue(std::move(packet));
        packet = Packet{};
    }
    if (!stopReceiver.load()) {
        disconnectedEvent = true;
        enqueue({0, 0, "server connection closed"});
    }
}
```

Listing 1: 客户端接收线程循环

接收线程持续调用 `recvPacket()` 函数，该函数先读取 12 字节报头，再根据报头中的长度字段读取消息体，确保接收完整数据包。

5.4 服务器初始运行界面

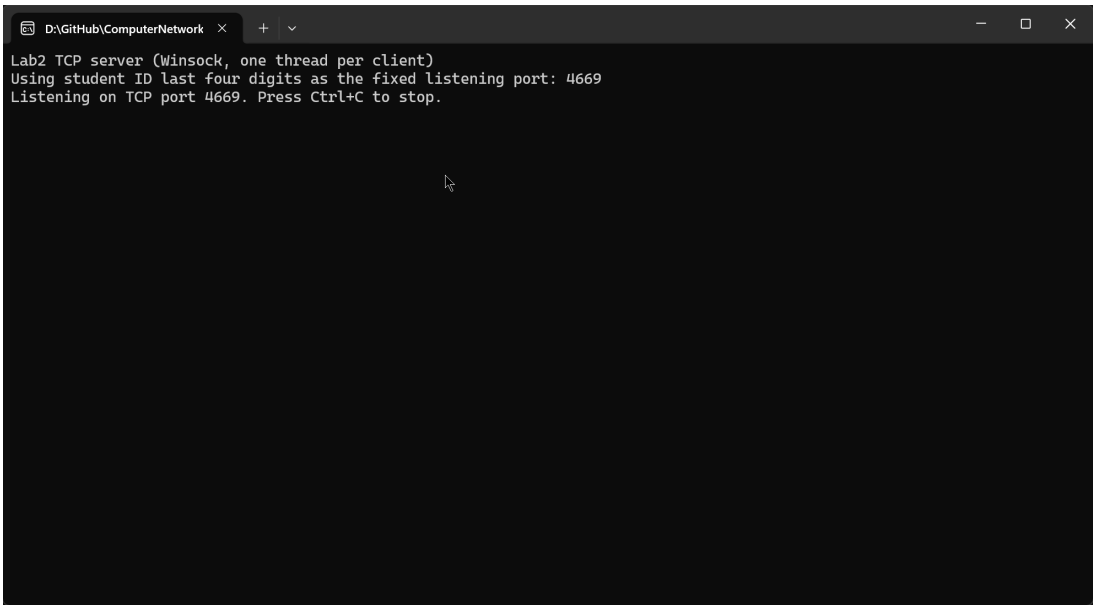


图 4: 服务器初始运行界面

服务器启动后显示监听端口 4669，并提示按 Ctrl+C 停止服务器。

5.5 服务器客户端处理子线程关键代码

服务器工作线程的核心代码如下（详见附录 B）：

```
void clientWorker(const std::shared_ptr<Client>& client) {
    sendToClient(client, {kGreeting, 0, "Connected to Lab2
        server"});

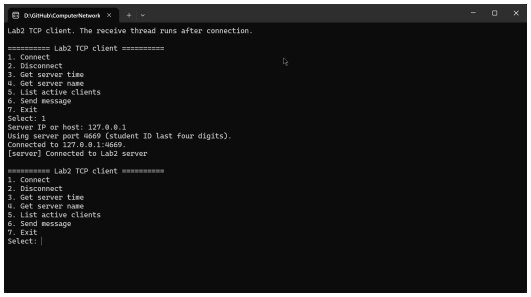
    Packet request;
    while (client->connected.load() && recvPacket(client->
        socket, request)) {
        switch (request.type) {
        case kRequestTime:
            // 处理时间请求
            break;
        case kRequestName:
            // 处理名字请求
            break;
        case kRequestList:
            // 处理列表请求
            break;
```

```
            case kRequestMessage:
                // 处理消息转发
                break;
            }
        }
        removeClient(client);
}
```

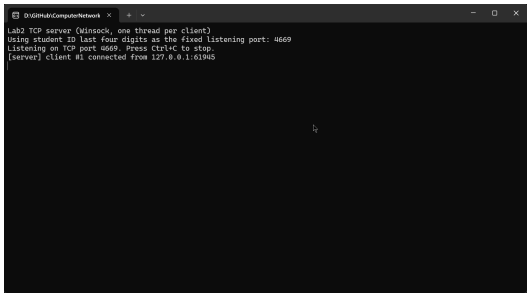
Listing 2: 服务器工作线程循环

每个客户端由独立线程处理，使用该客户端的 Socket 进行收发，不使用监听 Socket。

5.6 客户端选择连接功能



(a) 客户端

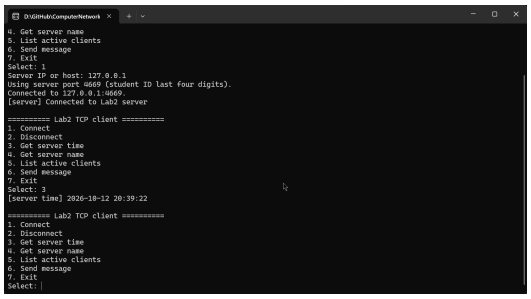


(b) 服务端

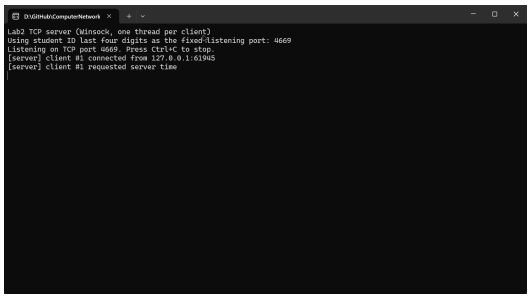
图 5: 客户端连接功能

客户端输入服务器 IP 地址（如 127.0.0.1），使用固定端口 4669 连接。连接成功后服务端显示客户端编号和地址信息。

5.7 客户端选择获取时间功能



(a) 客户端



(b) 服务端

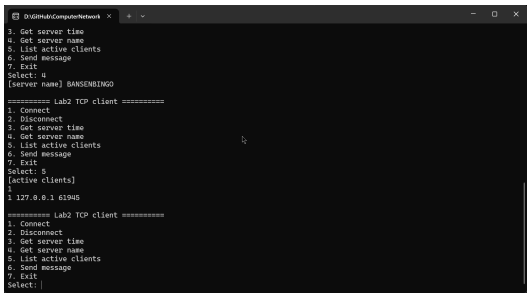
图 6: 获取服务器时间

服务端处理代码：

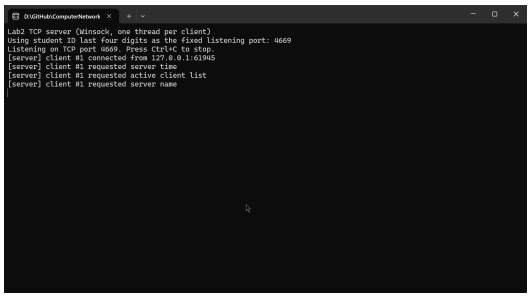
```cpp
case kRequestTime: {
    log("[server] client #" + std::to_string(client->id) + "
        requested server time");
    const std::time_t now = std::time(nullptr);
    std::tm localTime{};
    localtime_s(&localTime, &now);
    std::ostringstream timeText;
    timeText << std::put_time(&localTime, "%Y-%m-%d %H:%M:%S")
        ;
    sendToClient(client, {kResponseTime, request.requestId,
        timeText.str()});
    break;
}
```

Listing 3: 服务端处理时间请求

5.8 客户端选择获取名字功能



(a) 客户端



(b) 服务端

图 7: 获取服务器名字

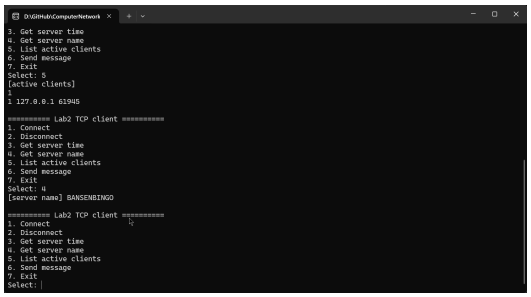
服务端处理代码：

```cpp
case kRequestName: {
    log("[server] client #" + std::to_string(client->id) + "
        requested server name");
    wchar_t name[MAX_COMPUTERNAME_LENGTH + 1]{};
    DWORD length = MAX_COMPUTERNAME_LENGTH + 1;
    if (!GetComputerNameW(name, &length)) {
        sendToClient(client, {kResponseError, request.
            requestId, "GetComputerName failed"});
        break;
    }
```

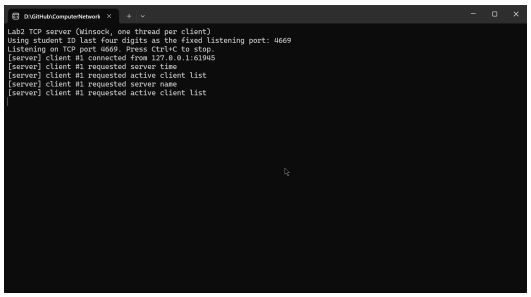
```
    const int size = WideCharToMultiByte(CP_UTF8, 0, name,
        static_cast<int>(length), nullptr, 0, nullptr, nullptr)
        ;
    std::string value(size, '\0');
    WideCharToMultiByte(CP_UTF8, 0, name, static_cast<int>(
        length), value.data(), size, nullptr, nullptr);
    sendToClient(client, {kResponseName, request.requestId,
        value});
    break;
}
```

Listing 4: 服务端处理名字请求

5.9 客户端选择获取客户端列表功能



(a) 客户端



(b) 服务端

图 8: 获取活动客户端列表

服务端处理代码：

```
std::string makeClientList() {
    std::vector<std::shared_ptr<Client>> snapshot;
    {
        std::lock_guard<std::mutex> lock(clientsMutex);
        for (const auto& [id, client] : clients) {
            if (client->connected.load()) snapshot.push_back(
                client);
        }
    }
    std::sort(snapshot.begin(), snapshot.end(),
              [](const auto& a, const auto& b) { return a->id
                  < b->id; });
    std::ostringstream result;
```

```
12      result << snapshot.size() << '\n';
13      for (const auto& client : snapshot) {
14          char ip[INET_ADDRSTRLEN]{};
15          inet_ntop(AF_INET, &client->address.sin_addr, ip,
                sizeof(ip));
16          result << client->id << ' ' << ip << ' ' << ntohs(
                client->address.sin_port) << '\n';
17      }
18      return result.str();
19  }
```

Listing 5: 服务端生成客户端列表

5.10 客户端选择发送消息功能

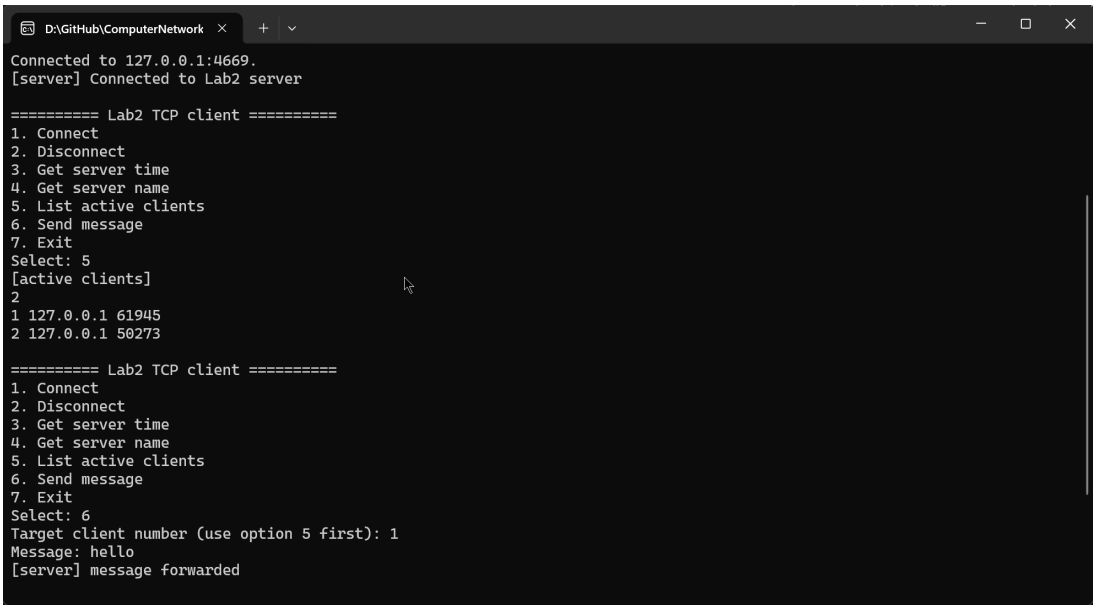


图 9: 发送消息的客户端

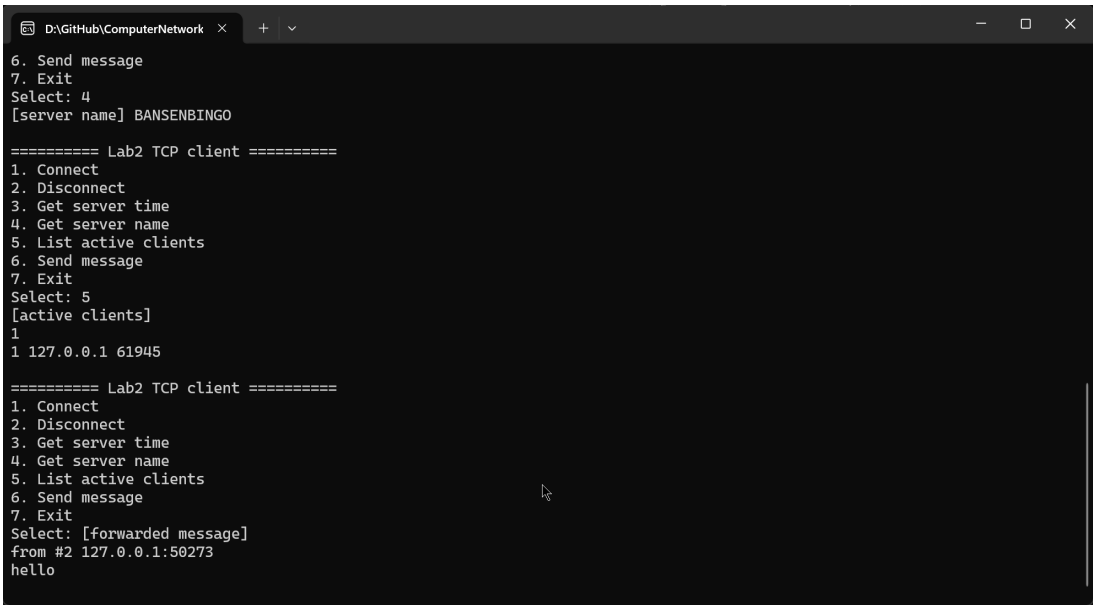


图 10: 接收消息的客户端

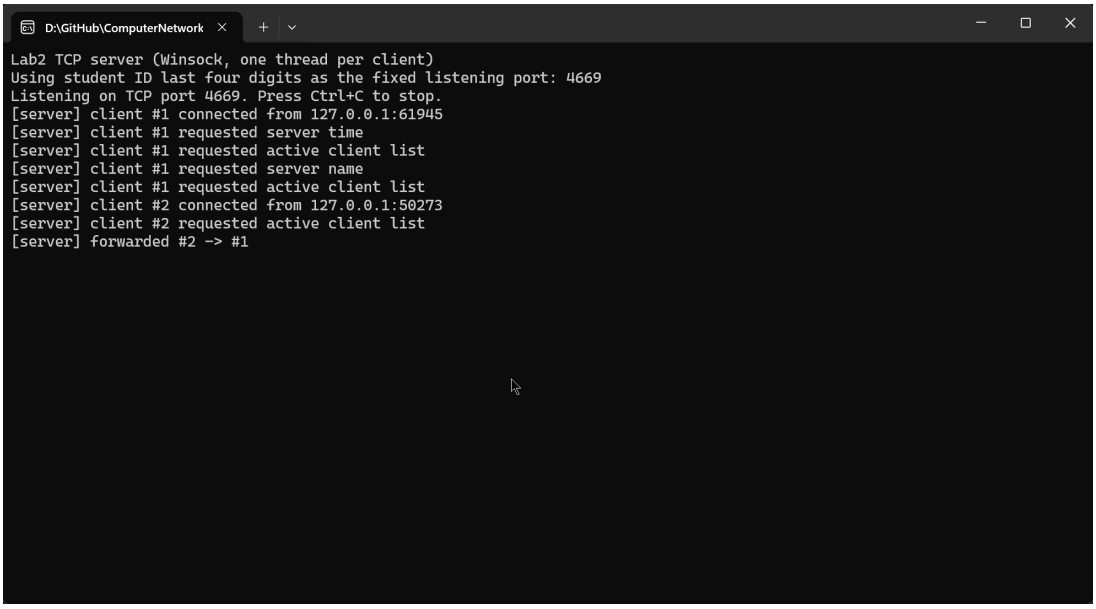


图 11: 服务端转发消息

服务端消息转发代码：

```
case kRequestMessage: {
    const auto separator = request.body.find('\n');
    int targetId = 0;
    if (!parsePositiveInt(request.body.substr(0, separator),
        targetId, 2147483647)) {
        sendToClient(client, {kResponseError, request.
            requestId, "invalid target client number"});
```

```
            break;
        }
        const std::string message = request.body.substr(separator
            + 1);

        std::shared_ptr<Client> target;
        {
            std::lock_guard<std::mutex> lock(clientsMutex);
            const auto it = clients.find(targetId);
            if (it != clients.end()) target = it->second;
        }

        if (!target || !target->connected.load()) {
            sendToClient(client, {kResponseError, request.
                requestId, "target client is not connected"});
            break;
        }

        std::ostringstream forwarded;
        forwarded << "from #" << client->id << ' ' << addressText(
            client->address) << "\n" << message;
        if (sendToClient(target, {kForwardMessage, 0, forwarded.
            str()})) {
            sendToClient(client, {kResponseMessageAck, request.
                requestId, "message forwarded"});
            log("[server] forwarded #" + std::to_string(client->id
                ) + " -> #" + std::to_string(targetId));
        }
        break;
    }
```

Listing 6: 服务端转发消息

客户端发送消息代码：

```
case 6: {
    if (!clientListFetched.load()) {
        std::cout << "Retrieve the active client list with
            option 5 before sending a message.\n";
        break;
    }
```

```
    const std::string target = readLineWithEvents("Target client number (use option 5 first): ");
    int targetId = 0;
    try { targetId = std::stoi(target); } catch (...) { targetId = 0; }
    const std::string message = readLineWithEvents("Message: ");
    waitForResponse(sendRequest(kRequestMessage, std::to_string(targetId) + "\n" + message));
    break;
}
```

Listing 7: 客户端发送消息

客户端接收消息代码：

```
case kForwardMessage:
    std::cout << "[forwarded message]\n" << packet.body << "\n";
    break;
```

Listing 8: 客户端处理转发消息

6 实验结果与分析

6.1 客户端是否需要调用 bind 操作？

客户端不需要调用 `bind()` 操作。客户端的源端口由操作系统自动分配。在调用 `connect()` 时，如果 Socket 未绑定端口，操作系统会从临时端口范围（通常是 49152-65535）中选择一个可用端口。

每次调用 `connect()` 时，如果使用新创建的 Socket，客户端的端口通常会变化，因为操作系统会分配一个新的临时端口。如果复用同一个 Socket，端口保持不变。

6.2 服务端 listen 和 accept 之间的断点测试

假设在服务端调用 `listen()` 和调用 `accept()` 之间设置断点，此时客户端调用 `connect()` 可以立即成功。这是因为 `listen()` 函数已经创建了连接等待队列，客户端的连接请求会被放入该队列中，即使服务端尚未调用 `accept()`。

`listen()` 的第二个参数指定了等待队列的最大长度（本程序使用 `SOMAXCONN`）。只要队列未满，客户端连接就能成功，但实际的数据传输要等到服务端调用 `accept()` 后才能进行。

6.3 服务器如何区分不同客户端的数据包？

服务器在同一个监听端口接收多个客户端的数据，通过不同的 Socket 句柄来区分数据包属于哪个客户端。

具体过程如下：

1. 服务器主线程在监听 Socket 上调用 `accept()`，每接受一个新连接，`accept()` 返回一个新的 Socket 句柄
2. 每个客户端对应一个唯一的 Socket 句柄和一个独立的工作线程
3. 工作线程只在自己的 Socket 句柄上调用 `recv()`，因此接收到的数据必然来自对应的客户端
4. 服务器维护一个客户端表，记录每个客户端的编号、Socket 句柄、IP 地址和端口

本程序中，每个客户端由 `Client` 结构体表示，包含唯一 ID 和 Socket 句柄，存储在全局的 `clients` 映射表中。

6.4 客户端主动断开后的 TCP 连接状态

客户端主动断开连接后，TCP 连接进入 `TIME_WAIT` 状态。该状态通常保持 2MSL（Maximum Segment Lifetime）时间，在 Windows 系统中通常是 120 秒（2 分钟）。

使用 `netstat -an` 命令可以观察到：

- 主动关闭方（客户端）进入 `TIME_WAIT` 状态
- 被动关闭方（服务端）先进入 `CLOSE_WAIT`，然后进入 `LAST_ACK`，最后关闭

`TIME_WAIT` 状态的目的是：

1. 确保最后的 ACK 能够到达对方
2. 确保旧连接的数据包在网络中消失，避免与新连接混淆

6.5 客户端异常退出时服务器的 TCP 状态变化

客户端断网后异常退出，服务器的 TCP 连接不会立即检测到断开，连接可能长时间保持在 `ESTABLISHED` 状态。这是因为 TCP 是双向独立的，一方断开不会立即通知另一方。

服务器检测连接是否有效的方法：

1. **调用 recv() 检测**：当连接断开时，`recv()` 返回 0 或 SOCKET_ERROR，这是本程序采用的方法

2. **设置 SO_KEEPALIVE 选项**：启用 TCP 保活机制，定期发送探测包

3. **应用层心跳**：定期发送心跳消息，超时未收到响应则认为连接断开

4. **设置接收超时**：使用 SO_RCVTIMEO 选项，超时后主动检测连接状态

本程序中，工作线程在 `recvPacket()` 中调用 `recv()`，如果客户端异常断开，`recv()` 会返回错误，工作线程检测到后调用 `removeClient()` 清理资源。

7 讨论与心得

7.1 实验过程中遇到的困难

1. **TCP 粘包和拆包问题**：TCP 是流式协议，一次 `send()` 可能被分成多个 TCP 段，一次 `recv()` 可能接收到多个消息或部分消息。通过实现 `sendAll()` 和 `recvExact()` 函数解决，确保发送和接收指定字节数。

2. **多线程并发控制**：多个线程同时访问客户端列表和 Socket 句柄需要加锁保护。使用 `std::mutex` 保护客户端列表的增删改查，使用每个客户端独立的 `sendMutex` 防止同时发送时数据交错。

3. **字节序转换**：网络传输使用大端序（网络字节序），而 Intel x86 架构使用小端序（主机字节序）。所有整数字段需要使用 `htonl()` 和 `ntohl()` 进行转换。

4. **客户端主线程阻塞问题**：如果主线程等待用户输入时使用阻塞式 `std::cin`，无法及时处理接收线程的消息。通过实现 `readLineWithEvents()` 函数，使用 `_kbhit()` 和 `_getch()` 非阻塞读取用户输入，同时定期调用 `drainMessages()` 处理接收到的消息。

5. **资源清理和线程安全退出**：服务端退出时需要正确关闭监听 Socket、通知所有工作线程退出、等待线程结束、关闭所有客户端 Socket。通过 `std::atomic` 标志和 `shutdown()` 函数实现优雅退出。

7.2 得到的经验教训

1. Socket 编程必须处理部分发送和部分接收，不能假定一次系统调用对应一个完整消息。

2. 多线程程序需要仔细设计并发控制，避免数据竞争和死锁。

3. 网络程序需要处理各种异常情况：连接断开、超时、对方异常退出等。

4. 使用自定义协议时，报文格式设计要考虑扩展性和健壮性，包括版本号、长度字段、类型字段等。

5. 测试时应该使用多个客户端同时连接，模拟并发场景，才能发现潜在的并发问题。

7.3 对本实验的建议

1. 建议提供更详细的协议设计示例，减少学生在协议设计上花费的时间。

2. 建议提供跨平台的实验环境，或者同时提供 Windows 和 Linux 的示例代码。

3. 建议增加性能测试要求，如测试最大并发连接数、消息吞吐量等。

4. 建议增加异常处理测试，如突然断网、服务端崩溃等场景。

A 附录 A：客户端完整源代码

```cpp
// Lab 2 client: interactive TCP client with a receive thread and message queue.
// Build with MinGW: g++ -std=c++17 -Wall -Wextra user.cpp -o user.exe -lws2_32

#define WIN32_LEAN_AND_MEAN
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <conio.h>
#include <atomic>
#include <cstdint>
#include <iostream>
#include <mutex>
#include <queue>
#include <string>
#include <thread>
#include <utility>
#include <chrono>

#ifdef _MSC_VER
#pragma comment(lib, "Ws2_32.lib")
#endif

namespace {
constexpr std::uint32_t kMaxBody = 64 * 1024;
constexpr int kServerPort = 4669; // server port from the student's ID
constexpr std::uint32_t kRequestTime = 1;
constexpr std::uint32_t kRequestName = 2;
constexpr std::uint32_t kRequestList = 3;
constexpr std::uint32_t kRequestMessage = 4;
constexpr std::uint32_t kResponseTime = 101;
constexpr std::uint32_t kResponseName = 102;
constexpr std::uint32_t kResponseList = 103;
constexpr std::uint32_t kForwardMessage = 104;
constexpr std::uint32_t kResponseMessageAck = 105;
constexpr std::uint32_t kGreeting = 106;
```

```
36 constexpr std::uint32_t kResponseError = 199;
37 constexpr int kHeaderSize = 12; // type, request id, body
       length
38
39 struct Packet {
40     std::uint32_t type = 0;
41     std::uint32_t requestId = 0;
42     std::string body;
43 };
44
45 bool sendAll(SOCKET socket, const char* data, int length) {
46     int sent = 0;
47     while (sent < length) {
48         const int count = send(socket, data + sent, length -
               sent, 0);
49         if (count == SOCKET_ERROR || count == 0) return false;
50         sent += count;
51     }
52     return true;
53 }
54
55 bool recvExact(SOCKET socket, char* data, int length) {
56     int received = 0;
57     while (received < length) {
58         const int count = recv(socket, data + received, length
               - received, 0);
59         if (count == SOCKET_ERROR || count == 0) return false;
60         received += count;
61     }
62     return true;
63 }
64
65 bool sendPacket(SOCKET socket, const Packet& packet) {
66     if (packet.body.size() > kMaxBody) return false;
67     std::uint32_t header[kHeaderSize / sizeof(std::uint32_t)]
           = {
68         htonl(packet.type), htonl(packet.requestId),
69         htonl(static_cast<std::uint32_t>(packet.body.size()))
               };
70     if (!sendAll(socket, reinterpret_cast<const char*>(header)
```

```
        , kHeaderSize)) return false;
    return packet.body.empty() || sendAll(socket, packet.body.
        data(), static_cast<int>(packet.body.size()));
}

bool recvPacket(SOCKET socket, Packet& packet) {
    std::uint32_t header[kHeaderSize / sizeof(std::uint32_t)
        ]{};
    if (!recvExact(socket, reinterpret_cast<char*>(header),
        kHeaderSize)) return false;
    if (ntohl(header[2]) > kMaxBody) return false;
    packet.type = ntohl(header[0]);
    packet.requestId = ntohl(header[1]);
    const auto bodyLength = ntohl(header[2]);
    packet.body.assign(bodyLength, '\0');
    return bodyLength == 0 || recvExact(socket, packet.body.
        data(), static_cast<int>(bodyLength));
}

struct QueuedMessage {
    Packet packet;
};

std::mutex queueMutex;
std::queue<QueuedMessage> messages;
std::atomic<bool> stopReceiver{false};
std::atomic<bool> connected{false};
std::atomic<bool> disconnectedEvent{false};
std::atomic<bool> clientListFetched{false};
std::atomic<std::uint32_t> waitingRequestId{0};
std::atomic<bool> waitingRequestDone{false};
SOCKET clientSocket = INVALID_SOCKET;
std::thread receiverThread;
std::uint32_t nextRequestId = 1;

void enqueue(Packet packet) {
    {
        std::lock_guard<std::mutex> lock(queueMutex);
        messages.push({std::move(packet)});
    }
```

```
106 }
107
108 void receiverLoop() {
109     Packet packet;
110     while (!stopReceiver.load() && recvPacket(clientSocket,
        packet)) {
111         enqueue(std::move(packet));
112         packet = Packet{};
113     }
114     if (!stopReceiver.load()) {
115         disconnectedEvent = true;
116         enqueue({0, 0, "server connection closed"});
117     }
118 }
119
120 void drainMessages() {
121     std::queue<QueuedMessage> local;
122     {
123         std::lock_guard<std::mutex> lock(queueMutex);
124         std::swap(local, messages);
125     }
126     while (!local.empty()) {
127         const Packet& packet = local.front().packet;
128         if (packet.requestId != 0 && packet.requestId ==
            waitingRequestId.load()) {
129             waitingRequestDone = true;
130         }
131         switch (packet.type) {
132         case kResponseTime:
133             std::cout << "[server time] " << packet.body << "\
                n";
134             break;
135         case kResponseName:
136             std::cout << "[server name] " << packet.body << "\
                n";
137             break;
138         case kResponseList:
139             std::cout << "[active clients]\n" << packet.body;
140             clientListFetched = true;
141             break;
```

```
142             case kForwardMessage:
143                 std::cout << "[forwarded message]\n" << packet.
                        body << "\n";
144                 break;
145             case kResponseMessageAck:
146                 std::cout << "[server] " << packet.body << "\n";
147                 break;
148             case kResponseError:
149                 std::cout << "[server error] " << packet.body << "
                        \n";
150                 break;
151             case kGreeting:
152                 std::cout << "[server] " << packet.body << "\n";
153                 break;
154             case 0:
155                 std::cout << "[connection] " << packet.body << "\n
                        ";
156                 break;
157             default:
158                 std::cout << "[server packet type=" << packet.type
                            << "] " << packet.body << "\n";
159                 break;
160             }
161             local.pop();
162         }
163         if (disconnectedEvent.exchange(false)) {
164             connected = false;
165             clientListFetched = false;
166         }
167 }
168
169 // Read a console line without blocking the main event loop.
        While the user is
170 // typing, the main thread continues draining messages from
        the receive thread.
171 std::string readLineWithEvents(const std::string& prompt) {
172     std::cout << prompt << std::flush;
173     std::string line;
174     while (true) {
175         drainMessages();
```

```
176         if (_kbhit()) {
177             const int first = _getch();
178             if (first == '\r' || first == '\n') {
179                 std::cout << '\n';
180                 return line;
181             }
182             if (first == '\b') {
183                 if (!line.empty()) {
184                     line.pop_back();
185                     std::cout << "\b \b" << std::flush;
186                 }
187                 continue;
188             }
189             if (first == 0 || first == 224) {
190                 // Consume the second byte of an extended key
                        (arrows, function keys, ...).
191                 (void)_getch();
192                 continue;
193             }
194             line.push_back(static_cast<char>(first));
195             std::cout << static_cast<char>(first) << std::
                    flush;
196         } else {
197             Sleep(30);
198         }
199     }
200 }
201
202 std::uint32_t sendRequest(std::uint32_t type, const std::
    string& body = std::string()) {
203     if (!connected.load()) {
204         std::cout << "Connect first.\n";
205         return 0;
206     }
207     const std::uint32_t requestId = nextRequestId++;
208     waitingRequestId = requestId;
209     waitingRequestDone = false;
210     const Packet request{type, requestId, body};
211     if (!sendPacket(clientSocket, request)) {
212         std::cerr << "send failed: " << WSAGetLastError() << "
```

```
            \n";
213         waitingRequestId = 0;
214         return 0;
215     }
216     return requestId;
217 }
218
219 void waitForResponse(std::uint32_t requestId) {
220     if (requestId == 0) return;
221     const auto deadline = std::chrono::steady_clock::now() +
            std::chrono::seconds(30);
222     while (connected.load() && !waitingRequestDone.load() &&
            std::chrono::steady_clock::now() < deadline) {
223         drainMessages();
224         Sleep(20);
225     }
226     drainMessages();
227     if (!waitingRequestDone.load() && connected.load()) {
228         std::cout << "Timed out waiting for the server
                response.\n";
229     }
230     waitingRequestId = 0;
231     waitingRequestDone = false;
232 }
233
234 void stopConnection() {
235     if (!connected.load() && !receiverThread.joinable())
            return;
236     stopReceiver = true;
237     if (clientSocket != INVALID_SOCKET) {
238         shutdown(clientSocket, SD_BOTH);
239     }
240     if (receiverThread.joinable()) receiverThread.join();
241     if (clientSocket != INVALID_SOCKET) {
242         closesocket(clientSocket);
243         clientSocket = INVALID_SOCKET;
244     }
245     connected = false;
246     clientListFetched = false;
247     stopReceiver = false;
```

```
248     disconnectedEvent = false;
249     waitingRequestId = 0;
250     waitingRequestDone = false;
251     std::cout << "Disconnected.\n";
252 }
253 
254 bool connectToServer() {
255     if (connected.load()) {
256         std::cout << "Already connected.\n";
257         return true;
258     }
259     const std::string host = readLineWithEvents("Server IP or
        host: ");
260     const int port = kServerPort;
261     std::cout << "Using server port " << port << " (student ID
        last four digits).\n";
262     clientSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
263     if (clientSocket == INVALID_SOCKET) {
264         std::cerr << "socket failed: " << WSAGetLastError() <<
            "\n";
265         return false;
266     }
267     sockaddr_in address{};
268     address.sin_family = AF_INET;
269     address.sin_port = htons(static_cast<u_short>(port));
270     if (inet_pton(AF_INET, host.c_str(), &address.sin_addr) !=
        1) {
271         addrinfo hints{};
272         hints.ai_family = AF_INET;
273         hints.ai_socktype = SOCK_STREAM;
274         addrinfo* result = nullptr;
275         if (getaddrinfo(host.c_str(), nullptr, &hints, &result
            ) != 0 || result == nullptr) {
276             std::cerr << "Could not resolve server address.\n"
                ;
277             closesocket(clientSocket);
278             clientSocket = INVALID_SOCKET;
279             return false;
280         }
281         address.sin_addr = reinterpret_cast<sockaddr_in*>(
```

```
            result->ai_addr)->sin_addr;
282         freeaddrinfo(result);
283     }
284     if (connect(clientSocket, reinterpret_cast<sockaddr*>(&
            address), sizeof(address)) == SOCKET_ERROR) {
285         std::cerr << "connect failed: " << WSAGetLastError()
                << "\n";
286         closesocket(clientSocket);
287         clientSocket = INVALID_SOCKET;
288         return false;
289     }
290     connected = true;
291     clientListFetched = false;
292     stopReceiver = false;
293     disconnectedEvent = false;
294     waitingRequestId = 0;
295     waitingRequestDone = false;
296     receiverThread = std::thread(receiverLoop);
297     std::cout << "Connected to " << host << ':' << port << ".\
            n";
298     return true;
299 }
300
301 void printMenu() {
302     std::cout << "\n========== Lab2 TCP client ==========" <<
            '\n'
303               << "1. Connect\n"
304               << "2. Disconnect\n"
305               << "3. Get server time\n"
306               << "4. Get server name\n"
307               << "5. List active clients\n"
308               << "6. Send message\n"
309               << "7. Exit\n";
310 }
311
312 } // namespace
313
314 int main() {
315     WSADATA wsaData{};
316     if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
```

```
317         std::cerr << "WSAStartup failed.\n";
318         return 1;
319     }
320     std::cout << "Lab2 TCP client. The receive thread runs
        after connection.\n";
321     bool quitting = false;
322     while (!quitting) {
323         drainMessages();
324         printMenu();
325         const std::string choiceText = readLineWithEvents("
            Select: ");
326         int choice = 0;
327         try { choice = std::stoi(choiceText); } catch (...) {
            choice = 0; }
328         if (!connected.load() && choice != 1 && choice != 7) {
329             std::cout << "Not connected: only Connect and Exit
                are available.\n";
330             continue;
331         }
332         switch (choice) {
333         case 1:
334             connectToServer();
335             break;
336         case 2:
337             stopConnection();
338             break;
339         case 3:
340             waitForResponse(sendRequest(kRequestTime));
341             break;
342         case 4:
343             waitForResponse(sendRequest(kRequestName));
344             break;
345         case 5:
346             waitForResponse(sendRequest(kRequestList));
347             break;
348         case 6: {
349             if (!clientListFetched.load()) {
350                 std::cout << "Retrieve the active client list
                    with option 5 before sending a message.\n";
351                 break;
```

```
352             }
353             const std::string target = readLineWithEvents("
                    Target client number (use option 5 first): ");
354             int targetId = 0;
355             try { targetId = std::stoi(target); } catch (...)
                    { targetId = 0; }
356             const std::string message = readLineWithEvents("
                    Message: ");
357             waitForResponse(sendRequest(kRequestMessage, std::
                    to_string(targetId) + "\n" + message));
358             break;
359         }
360         case 7:
361             stopConnection();
362             quitting = true;
363             break;
364         default:
365             std::cout << "Unknown menu choice.\n";
366             break;
367         }
368         // Allow queued responses and unsolicited forwarded
                messages to be shown promptly.
369         for (int i = 0; i < 5; ++i) {
370             Sleep(20);
371             drainMessages();
372         }
373     }
374     stopConnection();
375     WSACleanup();
376     return 0;
377 }
```

Listing 9: user.cpp

B 附录 B：服务端完整源代码

```cpp
// Lab 2 server: TCP socket API, one worker thread per connected client.
// Build with MinGW: g++ -std=c++17 -Wall -Wextra server.cpp -o server.exe -lws2_32
// Protocol: 12-byte network-order header (type, request_id, body_length), then UTF-8 text.

#define WIN32_LEAN_AND_MEAN
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <algorithm>
#include <atomic>
#include <cstdint>
#include <ctime>
#include <iomanip>
#include <iostream>
#include <memory>
#include <mutex>
#include <sstream>
#include <string>
#include <thread>
#include <unordered_map>
#include <vector>

#ifdef _MSC_VER
#pragma comment(lib, "Ws2_32.lib")
#endif

namespace {
constexpr std::uint32_t kMaxBody = 64 * 1024;
constexpr std::uint32_t kMaxMessage = kMaxBody - 256; // reserve space for sender metadata
constexpr int kServerPort = 4669; // student ID's last four digits
constexpr std::uint32_t kRequestTime = 1;
constexpr std::uint32_t kRequestName = 2;
constexpr std::uint32_t kRequestList = 3;
```

```
34 constexpr std::uint32_t kRequestMessage = 4;
35 constexpr std::uint32_t kResponseTime = 101;
36 constexpr std::uint32_t kResponseName = 102;
37 constexpr std::uint32_t kResponseList = 103;
38 constexpr std::uint32_t kForwardMessage = 104;
39 constexpr std::uint32_t kResponseMessageAck = 105;
40 constexpr std::uint32_t kGreeting = 106;
41 constexpr std::uint32_t kResponseError = 199;
42 constexpr int kHeaderSize = 12; // type, request id, body
       length
43
44 struct Packet {
45     std::uint32_t type = 0;
46     std::uint32_t requestId = 0;
47     std::string body;
48 };
49
50 bool sendAll(SOCKET socket, const char* data, int length) {
51     int sent = 0;
52     while (sent < length) {
53         const int count = send(socket, data + sent, length -
              sent, 0);
54         if (count == SOCKET_ERROR || count == 0) return false;
55         sent += count;
56     }
57     return true;
58 }
59
60 bool recvExact(SOCKET socket, char* data, int length) {
61     int received = 0;
62     while (received < length) {
63         const int count = recv(socket, data + received, length
              - received, 0);
64         if (count == SOCKET_ERROR || count == 0) return false;
65         received += count;
66     }
67     return true;
68 }
69
70 bool sendPacket(SOCKET socket, const Packet& packet) {
```

```
71      if (packet.body.size() > kMaxBody) return false;
72      std::uint32_t header[kHeaderSize / sizeof(std::uint32_t)]
            = {
73          htonl(packet.type), htonl(packet.requestId),
74          htonl(static_cast<std::uint32_t>(packet.body.size()))
                };
75      if (!sendAll(socket, reinterpret_cast<const char*>(header)
            , kHeaderSize)) return false;
76      return packet.body.empty() || sendAll(socket, packet.body.
            data(), static_cast<int>(packet.body.size()));
77  }
78
79  bool recvPacket(SOCKET socket, Packet& packet) {
80      std::uint32_t header[kHeaderSize / sizeof(std::uint32_t)
            ]{};
81      if (!recvExact(socket, reinterpret_cast<char*>(header),
            kHeaderSize)) return false;
82      if (ntohl(header[2]) > kMaxBody) return false;
83      packet.type = ntohl(header[0]);
84      packet.requestId = ntohl(header[1]);
85      const auto bodyLength = ntohl(header[2]);
86      packet.body.assign(bodyLength, '\0');
87      return bodyLength == 0 || recvExact(socket, packet.body.
            data(), static_cast<int>(bodyLength));
88  }
89
90  struct Client {
91      int id = 0;
92      SOCKET socket = INVALID_SOCKET;
93      sockaddr_in address{};
94      std::mutex sendMutex;
95      std::atomic<bool> connected{true};
96  };
97
98  std::mutex clientsMutex;
99  std::unordered_map<int, std::shared_ptr<Client>> clients;
100 int nextClientId = 1;
101 std::atomic<bool> serverRunning{true};
102 std::atomic<SOCKET> listenSocket{INVALID_SOCKET};
103 std::mutex logMutex;
```

```
104
105 bool parsePositiveInt(const std::string& text, int& value, int
        maximum) {
106     if (text.empty()) return false;
107     int number = 0;
108     for (const char digit : text) {
109         if (digit < '0' || digit > '9') return false;
110         const int next = digit - '0';
111         if (number > (maximum - next) / 10) return false;
112         number = number * 10 + next;
113     }
114     value = number;
115     return number > 0 && number <= maximum;
116 }
117
118 void log(const std::string& text) {
119     std::lock_guard<std::mutex> lock(logMutex);
120     std::cout << text << std::endl;
121 }
122
123 bool sendToClient(const std::shared_ptr<Client>& client, const
        Packet& packet) {
124     if (!client || packet.body.size() > kMaxBody) return false
           ;
125     std::lock_guard<std::mutex> lock(client->sendMutex);
126     // Recheck after acquiring the lock: the worker may have
           closed this socket.
127     if (!client->connected.load() || client->socket ==
           INVALID_SOCKET) return false;
128     if (sendPacket(client->socket, packet)) return true;
129     client->connected = false;
130     shutdown(client->socket, SD_BOTH); // wake its owning
           worker, which closes the socket
131     return false;
132 }
133
134 std::string addressText(const sockaddr_in& address) {
135     char ip[INET_ADDRSTRLEN]{};
136     inet_ntop(AF_INET, &address.sin_addr, ip, sizeof(ip));
137     std::ostringstream result;
```

```
138     result << ip << ':' << ntohs(address.sin_port);
139     return result.str();
140 }
141 
142 std::string makeClientList() {
143     std::vector<std::shared_ptr<Client>> snapshot;
144     {
145         std::lock_guard<std::mutex> lock(clientsMutex);
146         for (const auto& [id, client] : clients) {
147             if (client->connected.load()) snapshot.push_back(
                    client);
148         }
149     }
150     std::sort(snapshot.begin(), snapshot.end(), [](const auto&
            a, const auto& b) { return a->id < b->id; });
151     std::ostringstream result;
152     result << snapshot.size() << '\n';
153     for (const auto& client : snapshot) {
154         char ip[INET_ADDRSTRLEN]{};
155         inet_ntop(AF_INET, &client->address.sin_addr, ip,
                sizeof(ip));
156         result << client->id << ' ' << ip << ' ' << ntohs(
                client->address.sin_port) << '\n';
157     }
158     return result.str();
159 }
160 
161 void removeClient(const std::shared_ptr<Client>& client) {
162     if (!client) return;
163     client->connected = false;
164     {
165         std::lock_guard<std::mutex> sendLock(client->sendMutex
                );
166         shutdown(client->socket, SD_BOTH);
167         closesocket(client->socket);
168         client->socket = INVALID_SOCKET;
169     }
170     {
171         std::lock_guard<std::mutex> lock(clientsMutex);
172         clients.erase(client->id);
```

```
173     }
174     log("[server] client #" + std::to_string(client->id) + "
            disconnected (" + addressText(client->address) + ")");
175 }
176
177 void clientWorker(const std::shared_ptr<Client>& client) {
178     // A greeting is optional in the report; it also confirms
            that the worker owns this socket.
179     sendToClient(client, {kGreeting, 0, "Connected to Lab2
            server"});
180
181     Packet request;
182     while (client->connected.load() && recvPacket(client->
            socket, request)) {
183         if (request.requestId == 0 ||
184             (request.type != kRequestMessage && !request.body.
                empty())) {
185             sendToClient(client, {kResponseError, request.
                requestId, "invalid request id or body"});
186             continue;
187         }
188         switch (request.type) {
189         case kRequestTime: {
190             log("[server] client #" + std::to_string(client->
                id) + " requested server time");
191             const std::time_t now = std::time(nullptr);
192             std::tm localTime{};
193             localtime_s(&localTime, &now);
194             std::ostringstream timeText;
195             timeText << std::put_time(&localTime, "%Y-%m-%d %H
                :%M:%S");
196             sendToClient(client, {kResponseTime, request.
                requestId, timeText.str()});
197             break;
198         }
199         case kRequestName: {
200             log("[server] client #" + std::to_string(client->
                id) + " requested server name");
201             wchar_t name[MAX_COMPUTERNAME_LENGTH + 1]{};
202             DWORD length = MAX_COMPUTERNAME_LENGTH + 1;
```

```
203             if (!GetComputerNameW(name, &length)) {
204                 sendToClient(client, {kResponseError, request.
                        requestId, "GetComputerName failed"});
205                 break;
206             }
207             const int size = WideCharToMultiByte(CP_UTF8, 0,
                    name, static_cast<int>(length), nullptr, 0,
                    nullptr, nullptr);
208             std::string value(size, '\0');
209             WideCharToMultiByte(CP_UTF8, 0, name, static_cast<
                    int>(length), value.data(), size, nullptr,
                    nullptr);
210             sendToClient(client, {kResponseName, request.
                    requestId, value});
211             break;
212         }
213         case kRequestList:
214             log("[server] client #" + std::to_string(client->
                    id) + " requested active client list");
215             sendToClient(client, {kResponseList, request.
                    requestId, makeClientList()});
216             break;
217         case kRequestMessage: {
218             const auto separator = request.body.find('\n');
219             if (separator == std::string::npos) {
220                 sendToClient(client, {kResponseError, request.
                        requestId, "message request must be:
                        target_id\\nmessage"});
221                 break;
222             }
223             int targetId = 0;
224             if (!parsePositiveInt(request.body.substr(0,
                    separator), targetId, 2147483647)) {
225                 sendToClient(client, {kResponseError, request.
                        requestId, "invalid target client number"})
                        ;
226                 break;
227             }
228             const std::string message = request.body.substr(
                    separator + 1);
```

```
229             if (message.empty() || message.size() >
                    kMaxMessage) {
230                 sendToClient(client, {kResponseError, request.
                        requestId, "message must contain 1 to 65280
                         UTF-8 bytes"});
231                 break;
232             }
233             std::shared_ptr<Client> target;
234             {
235                 std::lock_guard<std::mutex> lock(clientsMutex)
                        ;
236                 const auto it = clients.find(targetId);
237                 if (it != clients.end()) target = it->second;
238             }
239             if (!target || !target->connected.load()) {
240                 sendToClient(client, {kResponseError, request.
                        requestId, "target client is not connected"
                        });
241                 break;
242             }
243             if (target->id == client->id) {
244                 sendToClient(client, {kResponseError, request.
                        requestId, "choose another connected client
                        "});
245                 break;
246             }
247             std::ostringstream forwarded;
248             forwarded << "from #" << client->id << ' ' <<
                    addressText(client->address) << "\n" << message
                    ;
249             if (sendToClient(target, {kForwardMessage, 0,
                    forwarded.str()})) {
250                 sendToClient(client, {kResponseMessageAck,
                        request.requestId, "message forwarded"});
251                 log("[server] forwarded #" + std::to_string(
                        client->id) + " -> #" + std::to_string(
                        targetId));
252             } else {
253                 sendToClient(client, {kResponseError, request.
                        requestId, "failed to send to target client
```

```
                    "});
254             }
255             break;
256         }
257         default:
258             sendToClient(client, {kResponseError, request.
                requestId, "unknown request type"});
259             break;
260         }
261     }
262     removeClient(client);
263 }
264
265 BOOL WINAPI consoleHandler(DWORD signal) {
266     if (signal == CTRL_C_EVENT || signal == CTRL_CLOSE_EVENT
        || signal == CTRL_BREAK_EVENT) {
267         serverRunning = false;
268         const SOCKET socket = listenSocket.exchange(
            INVALID_SOCKET);
269         if (socket != INVALID_SOCKET) {
270             shutdown(socket, SD_BOTH);
271             closesocket(socket);
272         }
273         return TRUE;
274     }
275     return FALSE;
276 }
277
278 } // namespace
279
280 int main() {
281     SetConsoleOutputCP(CP_UTF8);
282     SetConsoleCtrlHandler(consoleHandler, TRUE);
283     std::cout << "Lab2 TCP server (Winsock, one thread per
        client)\n";
284     const int port = kServerPort;
285     std::cout << "Using student ID last four digits as the
        fixed listening port: " << port << "\n";
286
287     WSADATA wsaData{};
```

```
288     if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
289         std::cerr << "WSAStartup failed.\n";
290         return 1;
291     }
292     const SOCKET serverSocket = socket(AF_INET, SOCK_STREAM,
            IPPROTO_TCP);
293     listenSocket = serverSocket;
294     if (serverSocket == INVALID_SOCKET) {
295         std::cerr << "socket failed: " << WSAGetLastError() <<
                "\n";
296         WSACleanup();
297         return 1;
298     }
299     BOOL exclusive = TRUE;
300     setsockopt(serverSocket, SOL_SOCKET, SO_EXCLUSIVEADDRUSE,
            reinterpret_cast<const char*>(&exclusive), sizeof(
            exclusive));
301     sockaddr_in serverAddress{};
302     serverAddress.sin_family = AF_INET;
303     serverAddress.sin_addr.s_addr = htonl(INADDR_ANY);
304     serverAddress.sin_port = htons(static_cast<u_short>(port))
            ;
305     if (bind(serverSocket, reinterpret_cast<sockaddr*>(&
            serverAddress), sizeof(serverAddress)) == SOCKET_ERROR
            ||
306         listen(serverSocket, SOMAXCONN) == SOCKET_ERROR) {
307         std::cerr << "bind/listen failed: " << WSAGetLastError
                () << "\n";
308         listenSocket = INVALID_SOCKET;
309         closesocket(serverSocket);
310         WSACleanup();
311         return 1;
312     }
313     std::cout << "Listening on TCP port " << port << ". Press
            Ctrl+C to stop.\n";
314
315     std::vector<std::thread> workers;
316     while (serverRunning.load()) {
317         sockaddr_in clientAddress{};
318         int addressLength = sizeof(clientAddress);
```

```
319         const SOCKET currentListenSocket = listenSocket.load()
                ;
320         if (currentListenSocket == INVALID_SOCKET) break;
321         SOCKET clientSocket = accept(currentListenSocket,
                reinterpret_cast<sockaddr*>(&clientAddress), &
                addressLength);
322         if (clientSocket == INVALID_SOCKET) {
323             if (serverRunning.load()) std::cerr << "accept
                    failed: " << WSAGetLastError() << "\n";
324             break;
325         }
326         if (!serverRunning.load()) {
327             closesocket(clientSocket);
328             break;
329         }
330         // Bound a blocked send so shutdown cannot wait
                indefinitely behind a sender.
331         DWORD sendTimeout = 5000;
332         setsockopt(clientSocket, SOL_SOCKET, SO_SNDTIMEO,
333                    reinterpret_cast<const char*>(&sendTimeout)
                           , sizeof(sendTimeout));
334         auto client = std::make_shared<Client>();
335         client->socket = clientSocket;
336         client->address = clientAddress;
337         {
338             std::lock_guard<std::mutex> lock(clientsMutex);
339             client->id = nextClientId++;
340             clients.emplace(client->id, client);
341         }
342         log("[server] client #" + std::to_string(client->id) +
                " connected from " + addressText(clientAddress));
343         try {
344             workers.emplace_back(clientWorker, client);
345         } catch (const std::exception& error) {
346             log(std::string("[server] could not start worker:
                    ") + error.what());
347             removeClient(client);
348         }
349     }
350
```

```
351     std::vector<std::shared_ptr<Client>> remainingClients;
352     {
353         std::lock_guard<std::mutex> lock(clientsMutex);
354         for (const auto& [id, client] : clients)
                remainingClients.push_back(client);
355     }
356     for (const auto& client : remainingClients) {
357         client->connected = false;
358         std::lock_guard<std::mutex> sendLock(client->sendMutex
                );
359         if (client->socket != INVALID_SOCKET) shutdown(client
                ->socket, SD_BOTH);
360     }
361     for (auto& worker : workers) if (worker.joinable()) worker
            .join();
362     const SOCKET remainingListenSocket = listenSocket.exchange
            (INVALID_SOCKET);
363     if (remainingListenSocket != INVALID_SOCKET) closesocket(
            remainingListenSocket);
364     WSACleanup();
365     return 0;
366 }
```

Listing 10: server.cpp